



Figure 3: Colourful patterns through ANSI escape sequences

interested in, like the colour of the text, and leave the text background and attribute. Also, the order of the parameter values in escape sequences does not matter—you can put the value of the background before that of colour, or the attribute before background, etc.

Text attribute	Text colour	Text background
0 normal	30 black	40 black
1 bold	31 red	41 red
4 underscore	32 green	42 green
7 reverse video	33 yellow	43 yellow
	34 blue	44 blue
	35 magenta	45 magenta
	36 cyan	46 cyan
	37 white	47 white

Table 2: ANSI escape sequence values

There are a lot of escape sequences—check the links in the *References* section—but not everything is guaranteed to work with *bash* and the GNU/Linux terminal; besides, some sequences are non-portable and obsolete. I verified that the values in Table 2 work on all my GNU/Linux distributions. Run the accompanying *testes.py* with *python testes.py* and see how these simple-looking escape sequences turn your console into a colourful graffiti wall.

The output of the script is shown in Figure 3. You can combine these concepts to tweak your shell prompts and turn your otherwise boring shell into a beautiful environment limited only by your imagination. Run *OPSI=\$PS1; PS1="\[033[31;1m<d \H @ \u \w> \033[m]"* in a text console, as an example. Please note that the ending *\033[m* is required, to reset the attributes after the prompt, else your command text will have the same attributes. You can also do terminal cursor movements through escape sequences like those in Table 3. Run the accompanying *curtest.sh* to see some of these, along with colour escape sequences, in action. I'll show a cleaner version of the text console cursor movements in a later section on *tput*.

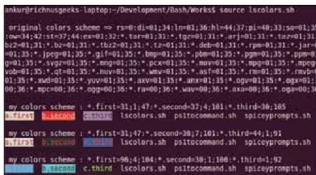


Figure 4: Directory listing with customised colours

Escape	Cursor movements
<code>\033[<y>:<x>H</code>	move cursor to x column, y line
<code>\033[<n>A</code>	move cursor n lines up
<code>\033[<n>B</code>	move cursor n line down
<code>\033[<n>C</code>	move cursor n columns forward
<code>\033[<n>D</code>	move cursor n columns backward
<code>\033[2J</code>	clear screen
<code>\033[K</code>	erase to end of line
<code>\033[s</code>	save cursor position
<code>\033[u</code>	restore cursor position

Table 3: Cursor movement sequences

Now let's play around with one more aspect before the next section. An environment variable dictates the colours of various entries when you run the *ls* command, with colour enabled in your terminal. Most GNU/Linux distributions have aliases set in *~/bashrc* to enable listings in colour in *bash*, and if this does not work, check the *ls* man page to find out how to enable it. The variable is *LS_COLORS*, and you can tweak it very easily. Run *dircolors* to see the code that sets the listing colours for your shell. (If *dircolors* is not available, run *echo \$LS_COLORS*.) The basic pattern in *LS_COLORS* is *type=[text colour;][text background;][text attribute]*. You can decode the contents of *LS_COLORS* based on information in Table 4, and any previous knowledge about escape sequence colour codes, with the additional colours in Table 5.

Content	Type
<i>di</i>	directory
<i>fi</i>	file
<i>ln</i>	symbolic link
<i>pi</i>	FIFO file
<i>so</i>	socket file
<i>bd</i>	block (buffered) special file
<i>cd</i>	character (unbuffered) special file
<i>or</i>	symbolic link pointing to a non-existent file (orphan)
<i>mi</i>	non-existent file referred by a symbolic link (visible when you type <i>ls -l</i>)
<i>ex</i>	executable file ('x' set in permission)
<i>*.ext</i>	file with a particular extension (e.g., *.c, *.cpp, *.ogg etc.)

Table 4: *LS_COLORS* settings